

Section 05 - Feature and Use Case List

Project: SalesOps AI

Prepared: May 24, 2026

Language: English

Source of truth: README.md, docs/*.md, backend/template.yaml, backend Lambda handlers, frontend React pages, and the public smoke script.

Section 5 Requirement Coverage

This document provides the full numbered feature and use-case list for the current SalesOps AI implementation. Each feature includes the actors, purpose, detailed story/script, system behavior, and source-code trace.

The feature IDs below can also be used as code-reference comments where the final submission requires code comments that identify which feature each code area implements.

Project Roles

- Rep: Sales or service representative who takes timed scenario exams and receives coaching.
- Manager: Admin/training manager who manages users, personas, scenarios, generated issues, and performance analytics.
- System: AWS serverless backend, Cognito, DynamoDB, SQS, Secrets Manager, OpenAI Responses API, and React frontend.

Feature Index

- F-01: Account registration and email confirmation.
- F-02: Sign in, protected session restore, token refresh, and sign out.
- F-03: Confirmation-code resend and forgot-password recovery.
- F-04: Role-based routing, navigation, and backend authorization.
- F-05: Manager user access administration.
- F-06: Persona library management.
- F-07: Scenario list, search, and filtering.
- F-08: Scenario draft creation and editing.
- F-09: Scenario publishing and readiness validation.
- F-10: AI issue generation with demo fallback.
- F-11: Generated issue review and editing.
- F-12: Scenario clone and archive.

- F-13: Rep scenario selection and timed exam start.
- F-14: Timed exam inbox, issue release pulse, and new-issue notification.
- F-15: Rep response submission and issue completion.
- F-16: Exam completion and AI evaluation creation.
- F-17: Rep results and coaching review.
- F-18: Manager performance dashboard, coaching queue, and CSV export.
- F-19: Operational health and smoke verification.

Detailed Features

F-01 - Account Registration and Email Confirmation

Actors: New rep, Cognito, backend auth service, DynamoDB.

Purpose: Create a new authenticated user account and store application profile data separately from credentials.

Main story/script:

1. Visitor opens /signup.
2. User enters full name, email, password, and password confirmation.
3. Frontend rejects mismatched passwords before contacting the API.
4. Frontend calls POST /auth/signup.
5. Backend creates the Cognito account and writes a DynamoDB profile.
6. New profile starts with role=rep.
7. If Cognito requires email verification, profile status is PENDING_CONFIRMATION.
8. User enters email confirmation code.
9. Frontend calls POST /auth/confirm.
10. Backend confirms the Cognito user and marks the DynamoDB profile ACTIVE.
11. User is sent to sign in.

System behavior and data:

- Cognito owns passwords, confirmation codes, and credential security.
- DynamoDB Users table stores userId, email, emailLower, fullName, role, status, createdAt, and updatedAt.
- Passwords are never stored in DynamoDB.

Code trace:

- Frontend: frontend/src/pages/SignupPage.tsx
- Frontend API: frontend/src/api/client.ts
- Backend: backend/src/handlers/auth.js (signup, confirm)

- Infrastructure: backend/template.yaml (SignUpFunction, ConfirmSignUpFunction, Cognito User Pool, UsersTable)
- Documentation: docs/auth-setup.md

F-02 - Sign In, Protected Session Restore, Token Refresh, and Sign Out

Actors: Rep, manager, Cognito, backend auth service.

Purpose: Allow active users to enter the app, keep sessions alive safely, restore sessions after refresh, and clear local access on sign out.

Main story/script:

1. User opens /login.
2. User enters email and password.
3. Frontend calls POST /auth/signin.
4. Backend authenticates through Cognito USER_PASSWORD_AUTH.
5. Backend decodes the ID token, finds or creates the DynamoDB profile, and rejects inactive users.
6. Frontend stores ID token, access token, refresh token, expiration timestamp, and user profile in local storage.
7. Frontend sets the ID token as the Authorization bearer token for API calls.
8. When the browser reloads, AuthProvider restores the saved session.
9. If the token is near expiration, frontend calls POST /auth/refresh.
10. Frontend calls GET /auth/me to validate the session and refresh profile data.
11. User signs out from the sidebar; frontend clears local storage and API authorization headers.

System behavior and data:

- ID tokens secure protected API Gateway routes.
- Refresh token extends the session without asking for a password again.
- Suspended users are blocked during sign in and profile refresh.

Code trace:

- Frontend: frontend/src/pages/LoginPage.tsx, frontend/src/auth/AuthContext.tsx, frontend/src/App.tsx
- Frontend API: frontend/src/api/client.ts
- Backend: backend/src/handlers/auth.js (signin, refresh, me)
- Infrastructure: backend/template.yaml (SignInFunction, RefreshFunction, MeFunction)

F-03 - Confirmation-Code Resend and Forgot-Password Recovery

Actors: User, Cognito, backend auth service.

Purpose: Support common account recovery cases without manual admin work.

Main story/script:

1. During signup confirmation, user can request another confirmation code.
2. Frontend calls POST /auth/resend-confirmation.
3. Backend asks Cognito to resend the signup confirmation code.
4. On login page, user chooses forgot password.
5. User enters email.
6. Frontend calls POST /auth/forgot-password.
7. Backend starts Cognito password recovery.
8. User enters reset code, new password, and password confirmation.
9. Frontend validates matching passwords.
10. Frontend calls POST /auth/forgot-password/confirm.
11. Backend confirms the new password with Cognito.
12. User returns to sign in with the new password.

System behavior and data:

- Recovery uses verified email through Cognito.
- Backend maps Cognito errors to clear user-facing messages.
- No password reset data is stored in DynamoDB.

Code trace:

- Frontend: frontend/src/pages/SignupPage.tsx, frontend/src/pages/LoginPage.tsx
- Frontend API: frontend/src/api/client.ts
- Backend: backend/src/handlers/auth.js (resendConfirmation, forgotPassword, confirmForgotPassword)
- Infrastructure: backend/template.yaml (ResendConfirmationFunction, ForgotPasswordFunction, ConfirmForgotPasswordFunction)

F-04 - Role-Based Routing, Navigation, and Backend Authorization

Actors: Rep, manager, frontend route guards, API Gateway Cognito authorizer, backend content service.

Purpose: Show each user only the routes and actions allowed for their role, and enforce the same rules on the backend.

Main story/script:

1. Anonymous user tries to open a protected route.
2. Frontend route guard redirects the user to /login.
3. Signed-in rep sees the exam route only.
4. Signed-in manager sees dashboard, users, personas, and scenarios.

5. If a rep tries manager route URLs manually, frontend redirects to `/exam/start`.
6. If a manager tries rep exam URLs, frontend redirects to `/dashboard`.
7. API Gateway requires Cognito authorization for protected endpoints.
8. Backend checks the DynamoDB profile before manager-only or rep-only work.
9. Backend rejects inactive users, wrong roles, missing users, and missing authenticated claims.

System behavior and data:

- Frontend route guards improve user experience.
- Backend `requireManager`, `requireRep`, and `requireActiveUser` are the real security controls.
- Manager APIs return 403 for reps and suspended users.

Code trace:

- Frontend: `frontend/src/App.tsx`
- Backend: `backend/src/handlers/content.js` (`requireManager`, `requireActiveUser`, `requireRep`)
- Infrastructure: `backend/template.yaml` Cognito authorizer on protected routes
- Smoke verification: `scripts/smoke.mjs`

F-05 - Manager User Access Administration

Actors: Manager, users table, backend content service.

Purpose: Let managers see user profiles and control application-level role and status.

Main story/script:

1. Manager opens `/users`.
2. Frontend loads users through `GET /users`.
3. Manager searches by name, email, or user ID.
4. Manager filters by role and status.
5. Manager opens a user edit modal.
6. Manager changes role to rep or manager.
7. Manager changes status to `ACTIVE` or `SUSPENDED`.
8. Frontend calls `PUT /users/{userId}`.
9. Backend validates manager permission, valid role, valid status, and existing user.
10. Backend blocks role/status edits for `PENDING_CONFIRMATION` users.
11. Backend blocks a manager from demoting or suspending their own account.
12. Backend saves the updated DynamoDB user profile.
13. Frontend refreshes the user table.

System behavior and data:

- This is profile-level access control; Cognito credentials remain unchanged.
- Suspended users cannot sign in or access protected routes.
- First manager can be promoted directly in DynamoDB as documented.

Code trace:

- Frontend: `frontend/src/pages/UsersPage.tsx`
- Frontend API: `frontend/src/api/client.ts`
- Backend: `backend/src/handlers/content.js` (`listUsers`, `updateUser`)
- Infrastructure: `backend/template.yaml` (`ListUsersFunction`, `UpdateUserFunction`, `UsersTable`)
- Documentation: `docs/auth-setup.md`

F-06 - Persona Library Management

Actors: Manager, backend content service, DynamoDB Personas table.

Purpose: Let managers create reusable customer behavior profiles that drive scenario issue generation.

Main story/script:

1. Manager opens `/personas`.
2. Frontend loads personas through `GET /personas`.
3. Page shows active count, other count, and total count.
4. Manager searches across name, description, and behavior notes.
5. Manager filters by status.
6. Manager clicks Add persona.
7. Manager enters name, description, and behavior notes.
8. Frontend calls `POST /personas`.
9. Backend validates required name and writes a persona with ACTIVE status.
10. Manager can click an existing persona row or edit button.
11. Manager updates name, description, or behavior notes.
12. Frontend calls `PUT /personas/{personaId}`.
13. Backend validates the persona exists and saves the update.
14. Frontend refreshes the table.

System behavior and data:

- Personas are manager-only.
- Personas are referenced by scenarios through persona IDs.
- Behavior notes help the LLM create realistic customer messages.

Code trace:

- Frontend: `frontend/src/pages/PersonasPage.tsx`
- Frontend API: `frontend/src/api/client.ts`
- Backend: `backend/src/handlers/content.js` (`listPersonas`, `createPersona`, `updatePersona`)
- Infrastructure: `backend/template.yaml` (`PersonasTable`, `persona` functions)

F-07 - Scenario List, Search, and Filtering

Actors: Manager, scenario library, persona library.

Purpose: Give managers an operational workbench for finding and reviewing all scenario content.

Main story/script:

1. Manager opens `/scenarios`.
2. Frontend loads personas and scenarios.
3. Page shows counts for published, draft, archived, and total scenarios.
4. Manager searches by scenario title, description, or linked persona name.
5. Manager filters by active/all/status.
6. Manager filters by persona.
7. Scenario table shows title, status, personas, issue target, generated issue count, and last update time.
8. Manager clicks a row to open scenario details.
9. Detail modal shows summary, personas, readiness checklist, generated issues, and scenario actions.

System behavior and data:

- Scenario search is frontend-side over loaded scenario and persona data.
- Archived scenarios can be hidden from normal manager view but still inspected when filtering all status.

Code trace:

- Frontend: `frontend/src/pages/ScenarioBuilderPage.tsx`
- Frontend API: `frontend/src/api/client.ts`
- Backend: `backend/src/handlers/content.js` (`listScenarios`)
- Infrastructure: `backend/template.yaml` (`ListScenariosFunction`, `ScenariosTable`)

F-08 - Scenario Draft Creation and Editing

Actors: Manager, persona library, backend content service.

Purpose: Let managers define training scenarios before they become available for reps.

Main story/script:

1. Manager clicks Add scenario.

2. Frontend opens a scenario form modal.
3. Manager enters title and description.
4. Manager selects one persona from existing personas.
5. Manager sets planned issue count from 1 to 20.
6. Manager clicks Save draft.
7. Frontend calls POST /scenarios for new scenario or PUT /scenarios/{scenarioId} for existing scenario.
8. Backend validates title and issue count.
9. Backend stores or updates scenario as DRAFT.
10. Frontend refreshes scenario list and selects the saved scenario.

System behavior and data:

- Scenario records store title, description, persona IDs, issue count, generated issues, status, timestamps, generation source, and generation warning.
- Issue count is bounded from 1 to 20 to keep exam scope controlled.

Code trace:

- Frontend: frontend/src/pages/ScenarioBuilderPage.tsx
- Frontend API: frontend/src/api/client.ts
- Backend: backend/src/handlers/content.js (createScenario, updateScenario, parseIssueCount)
- Infrastructure: backend/template.yaml (CreateScenarioFunction, UpdateScenarioFunction)

F-09 - Scenario Publishing and Readiness Validation

Actors: Manager, backend content service.

Purpose: Move a scenario from draft into a published state and make readiness visible before reps use it.

Main story/script:

1. Manager creates or edits a scenario.
2. Manager clicks Publish from the scenario form.
3. Frontend saves the current form data first.
4. Frontend calls POST /scenarios/{scenarioId}/publish.
5. Backend validates that the scenario exists.
6. Backend validates at least one persona is selected.
7. Backend changes scenario status to PUBLISHED.
8. Scenario detail modal shows readiness checklist:
 - Persona selected.
 - Published.
 - Issues generated.

- Issue count matched.
- Ready for reps.
- 9. Scenario is still not visible to reps until it has generated issues.

System behavior and data:

- Publish is manager-only.
- Backend prevents publishing scenarios without personas.
- Rep scenario list filters to PUBLISHED scenarios with at least one generated issue.

Code trace:

- Frontend: `frontend/src/pages/ScenarioBuilderPage.tsx` (`handlePublish`, `readinessItems`)
- Frontend API: `frontend/src/api/client.ts`
- Backend: `backend/src/handlers/content.js` (`publishScenario`, `listExamScenarios`)
- Infrastructure: `backend/template.yaml` (`PublishScenarioFunction`, `ListExamScenariosFunction`)

F-10 - AI Issue Generation with Demo Fallback

Actors: Manager, OpenAI Responses API, AWS Secrets Manager, backend content service.

Purpose: Generate realistic inbox issues for a published scenario, while keeping the lab workflow usable if the LLM is unavailable.

Main story/script:

1. Manager opens a published scenario detail modal.
2. Manager clicks Generate issues or Regenerate issues.
3. Frontend calls `POST /scenarios/{scenarioId}/issues/generate`.
4. Backend validates manager role, existing scenario, published status, selected personas, and existing persona records.
5. Backend reads `OPENAI_API_KEY` from AWS Secrets Manager secret `salesops/dev/llm-api-keys`.
6. Backend sends scenario, persona data, and exact issue count to OpenAI Responses API using model `gpt-5-mini` by default.
7. Backend requests strict JSON schema output with one issue per planned count.
8. Backend validates generated persona IDs, required fields, and difficulty values.
9. Backend stores generated issues on the scenario with `generationSource=OPENAI`.
10. If OpenAI or secret access fails with a server-side error, backend creates clearly marked demo issues instead.
11. Backend stores `generationSource=DEMO` and a warning message.
12. Frontend shows the generation warning if fallback was used.

System behavior and data:

- LLM key is not stored in frontend code.

- Generated issues contain `issueId`, `personaId`, `customerName`, `subject`, `message`, `difficulty`, `status`, `createdAt`, and `updatedAt`.
- Demo fallback keeps exam flow testable even without OpenAI availability.

Code trace:

- Frontend: `frontend/src/pages/ScenarioBuilderPage.tsx`
- Frontend API: `frontend/src/api/client.ts`
- Backend: `backend/src/handlers/content.js` (`generateScenarioIssues`, `requestGeneratedIssues`, `generateIssuesWithFallback`, `buildDemoIssues`)
- Infrastructure: `backend/template.yaml` (`GenerateScenarioIssuesFunction`, `LlmSecretName`, `OpenAiModel`)
- Documentation: `docs/aws-lab-setup.md`

F-11 - Generated Issue Review and Editing

Actors: Manager, scenario library, backend content service.

Purpose: Let managers control generated exam material before reps see it.

Main story/script:

1. Manager opens a scenario detail modal.
2. Generated issues appear as editable cards.
3. Manager reviews persona, customer name, difficulty, subject, and message.
4. Manager edits customer name, subject, message, or difficulty.
5. Manager clicks Save issue.
6. Frontend calls `PUT /scenarios/{scenarioId}/issues/{issueId}`.
7. Backend validates manager role, scenario existence, issue existence, required customer name, subject, message, and difficulty.
8. Backend updates only the selected issue inside the scenario record.
9. Frontend refreshes the scenario and shows saved issue data.

System behavior and data:

- Difficulty must be EASY, MEDIUM, or HARD.
- Editing generated content allows human approval of LLM-created material.

Code trace:

- Frontend: `frontend/src/pages/ScenarioBuilderPage.tsx` (`issueDrafts`, `handleSaveIssue`)
- Frontend API: `frontend/src/api/client.ts`
- Backend: `backend/src/handlers/content.js` (`updateScenarioIssue`, `validateDifficulty`)
- Infrastructure: `backend/template.yaml` (`UpdateScenarioIssueFunction`)

F-12 - Scenario Clone and Archive

Actors: Manager, scenario library, backend content service.

Purpose: Support reuse of good scenarios and remove old scenarios from rep availability without deleting history.

Main story/script:

1. Manager opens scenario details.
2. Manager clicks Clone.
3. Frontend calls `POST /scenarios/{scenarioId}/clone`.
4. Backend copies scenario data into a new scenario with new scenario ID.
5. Backend gives copied issues new issue IDs.
6. New scenario is saved as DRAFT.
7. Manager can edit the clone without changing the original.
8. Manager can click Archive on an existing scenario.
9. Frontend asks for confirmation.
10. Frontend calls `POST /scenarios/{scenarioId}/archive`.
11. Backend changes scenario status to ARCHIVED.
12. Archived scenario is hidden from rep scenario list.

System behavior and data:

- Clone preserves content but creates fresh IDs.
- Archive keeps record history and removes scenario from normal rep exam availability.

Code trace:

- Frontend: `frontend/src/pages/ScenarioBuilderPage.tsx` (`handleCloneScenario`, `handleArchiveScenario`)
- Frontend API: `frontend/src/api/client.ts`
- Backend: `backend/src/handlers/content.js` (`cloneScenario`, `archiveScenario`)
- Infrastructure: `backend/template.yaml` (`CloneScenarioFunction`, `ArchiveScenarioFunction`)

F-13 - Rep Scenario Selection and Timed Exam Start

Actors: Rep, backend content service, DynamoDB ExamSessions table, SQS.

Purpose: Let reps start a timed exam from a manager-approved scenario.

Main story/script:

1. Rep opens `/exam/start`.
2. Frontend calls `GET /exam/scenarios`.

3. Backend verifies active rep role.
4. Backend returns only published scenarios with generated issues.
5. Rep selects a scenario.
6. Brief panel shows title, description, and exam duration.
7. Rep clicks Start.
8. Frontend calls `POST /exam/sessions` with selected scenario ID.
9. Backend validates scenario existence, published status, and generated issues.
10. Backend creates exam metadata record in DynamoDB.
11. Backend creates one issue record per generated scenario issue.
12. First issue is visible immediately.
13. Later issues receive release times across the 3-minute exam.
14. Backend schedules delayed SQS messages for later issue release.
15. Frontend navigates to `/exam/{sessionId}`.

System behavior and data:

- Exam duration is 180 seconds.
- Session data uses DynamoDB partition key `sessionId` and record IDs for metadata, issue records, and evaluation.
- Only reps can create exam sessions.

Code trace:

- Frontend: `frontend/src/pages/ExamStartPage.tsx`
- Frontend API: `frontend/src/api/client.ts`
- Backend: `backend/src/handlers/content.js` (`listExamScenarios`, `createExamSession`, `releaseDelaySeconds`, `scheduleIssueRelease`)
- Infrastructure: `backend/template.yaml` (`ListExamScenariosFunction`, `CreateExamSessionFunction`, `ExamSessionsTable`, `ExamIssueReleaseQueue`)

F-14 - Timed Exam Inbox, Issue Release Pulse, and New-Issue Notification

Actors: Rep, frontend exam page, backend content service, SQS release Lambda.

Purpose: Simulate a live customer inbox where issues arrive over time during the exam.

Main story/script:

1. Rep enters `/exam/{sessionId}`.
2. Frontend calls `GET /exam/sessions/{sessionId}/pulse` every 2 seconds.
3. Backend verifies the rep owns the session.
4. Backend reveals any due issues whose release time has passed.
5. Backend returns session metadata, remaining time, status, and visible issues.
6. Frontend shows an inbox list with issue subject, customer name, difficulty, and done state.

7. Frontend automatically selects the first visible issue.
8. When a new issue arrives, frontend shows a toast with the issue subject.
9. The countdown timer updates every second.
10. When remaining time reaches zero, frontend locks response entry and shows end-of-exam overlay.
11. SQS-triggered `releaseExamIssue` also marks scheduled issues visible when delay expires.

System behavior and data:

- Polling plus SQS delayed release makes issue arrival resilient.
- Backend ownership checks prevent reps from reading other reps' sessions.
- Issues hidden until release time cannot be answered early.

Code trace:

- Frontend: `frontend/src/pages/ExamPage.tsx`
- Frontend API: `frontend/src/api/client.ts`
- Backend: `backend/src/handlers/content.js` (`getExamSessionPulse`, `revealDueExamIssues`, `releaseExamIssue`, `markExamIssueVisible`)
- Infrastructure: `backend/template.yaml` (`GetExamSessionPulseFunction`, `ReleaseExamIssueFunction`, SQS event source)

F-15 - Rep Response Submission and Issue Completion

Actors: Rep, backend content service, DynamoDB ExamSessions table.

Purpose: Capture representative answers for each visible customer issue and allow the rep to mark an issue done.

Main story/script:

1. Rep selects a visible issue in the exam inbox.
2. Customer message appears in chat-style thread.
3. Rep drafts a response.
4. Frontend disables response entry if exam ended or issue is already done.
5. Rep clicks Submit response.
6. Frontend calls `POST /exam/sessions/{sessionId}/issues/{issueId}/responses`.
7. Backend verifies rep role, session ownership, active session, visible issue, not done issue, non-empty response, and max length 4000 characters.
8. Backend appends the response with generated response ID and timestamp.
9. Frontend updates the chat thread.
10. Rep can submit multiple responses for the same issue while the issue is open.
11. Rep clicks Done.
12. Frontend calls `POST /exam/sessions/{sessionId}/issues/{issueId}/done`.
13. Backend requires at least one response before done.

14. Backend marks the issue DONE and stores doneAt.

System behavior and data:

- Responses are stored inside each issue record as an ordered list.
- Done issues are locked from additional response submission.
- Backend prevents response after exam end.

Code trace:

- Frontend: frontend/src/pages/ExamPage.tsx
- Frontend API: frontend/src/api/client.ts
- Backend: backend/src/handlers/content.js (submitExamIssueResponse, markExamIssueDone, getOwnedExamIssue)
- Infrastructure: backend/template.yaml (SubmitExamIssueResponseFunction, MarkExamIssueDoneFunction)

F-16 - Exam Completion and AI Evaluation Creation

Actors: Rep, backend content service, OpenAI Responses API, AWS Secrets Manager.

Purpose: Score completed exams and create practical coaching output.

Main story/script:

1. Exam timer reaches zero.
2. Frontend shows Exam Ended overlay.
3. Rep clicks Continue to Evaluation.
4. Frontend calls POST /exam/sessions/{sessionId}/evaluation.
5. Backend verifies rep ownership.
6. Backend returns existing evaluation if already created.
7. Backend rejects evaluation while exam is still active.
8. Backend loads all exam issue records and rep responses.
9. Backend reads OpenAI key from Secrets Manager.
10. Backend sends scenario, customer issues, rep responses, and rubric to OpenAI.
11. Rubric weights are kindness 25%, professionalism 25%, resolution 25%, clarity 15%, and helpful ideas 10%.
12. Backend requests strict JSON schema evaluation with rubric scores, notes, strengths, growth areas, practice ideas, and per-issue feedback.
13. Backend normalizes 1-5 scores if returned, clamps scores to 0-100, and computes weighted final score.
14. Backend stores evaluation record in DynamoDB and marks session ENDED.
15. Frontend navigates to /exam/{sessionId}/results.

System behavior and data:

- Evaluation is persisted and can be fetched later.
- Missing rep responses receive low scores and coaching notes.
- Multiple responses for one issue are evaluated together.

Code trace:

- Frontend: frontend/src/pages/ExamPage.tsx
- Frontend API: frontend/src/api/client.ts
- Backend: backend/src/handlers/content.js (createExamEvaluation, requestExamEvaluation, normalizeExamEvaluation, weightedEvaluationScore)
- Infrastructure: backend/template.yaml (CreateExamEvaluationFunction, LlmSecretName, OpenAiModel)

F-17 - Rep Results and Coaching Review

Actors: Rep, backend content service, persisted evaluation.

Purpose: Show final exam score and actionable coaching after evaluation is complete.

Main story/script:

1. Rep opens /exam/{sessionId}/results.
2. Frontend calls GET /exam/sessions/{sessionId}/evaluation.
3. Backend verifies rep owns the session.
4. Backend returns stored evaluation.
5. Results page shows final score out of 100.
6. Page shows rubric meters for kindness, professionalism, resolution, clarity, and helpful ideas.
7. Page shows AI notes, strengths, growth areas, and practice ideas.
8. Page shows per-issue coaching with issue score, notes, and suggested answer ideas.
9. Rep can start a new exam from results page.
10. If evaluation is missing, page shows a not-ready state and link back to exam.

System behavior and data:

- Results are read-only to reps.
- Evaluation endpoint does not recalculate by default; it returns saved result.

Code trace:

- Frontend: frontend/src/pages/ExamResultsPage.tsx
- Frontend API: frontend/src/api/client.ts
- Backend: backend/src/handlers/content.js (getExamEvaluation, itemToExamEvaluation)
- Infrastructure: backend/template.yaml (GetExamEvaluationFunction)

F-18 - Manager Performance Dashboard, Coaching Queue, and CSV Export

Actors: Manager, backend content service, exam data, user data, scenario data.

Purpose: Give managers visibility into rep performance, evaluation coverage, pass rate, and coaching focus.

Main story/script:

1. Manager opens /dashboard.
2. Frontend calls GET /dashboard with selected scenario filter, default ALL.
3. Backend verifies manager role.
4. Backend scans exam sessions, users, and scenarios.
5. Backend groups exam records by session ID.
6. Backend detects active, completed, evaluated, and not-evaluated attempts.
7. Backend calculates summary metrics: total attempts, active attempts, completed attempts, evaluated attempts, average success score, pass rate, rep count, reps evaluated, and needs evaluation.
8. Backend builds scenario summaries with attempts, average score, and pass rate.
9. Backend builds score bands: Passed, Needs coaching, At risk, and Not evaluated.
10. Backend builds rep rows with attempts, latest score, average score, best score, pass rate, completion rate, evaluated attempts, needs evaluation, last attempt date, and coaching focus.
11. Frontend shows metric strip, pie chart or column chart, coaching queue, and sortable rep roster.
12. Manager filters dashboard by scenario.
13. Manager switches chart mode between pie and columns.
14. Manager refreshes dashboard data.
15. Manager exports rep rows to CSV named salesops-dashboard-reps.csv.

System behavior and data:

- Pass score is 80.
- Coaching queue prioritizes reps needing evaluation or below pass score.
- Dashboard reads existing operational data; it does not modify records.

Code trace:

- Frontend: frontend/src/pages/DashboardPage.tsx
- Frontend API: frontend/src/api/client.ts
- Backend: backend/src/handlers/content.js (getDashboard, dashboard helper functions)
- Infrastructure: backend/template.yaml (GetDashboardFunction)

F-19 - Operational Health and Smoke Verification

Actors: Developer, system administrator, AWS API Gateway, Lambda health endpoint.

Purpose: Verify deployment wiring, public health availability, protected route security, and basic auth validation.

Main story/script:

1. Developer deploys backend with SAM.
2. Developer sets frontend API base URL through VITE_API_BASE_URL.
3. Developer runs `npm run smoke`.
4. Smoke script reads API base URL from environment or `frontend/.env.local`.
5. Smoke script calls `GET /health`.
6. Health endpoint returns service name, stage, timestamp, and status ok.
7. Smoke script calls protected endpoints without credentials.
8. Protected `/personas`, `/exam/scenarios`, and `/dashboard` must reject anonymous requests with 401.
9. Smoke script sends empty sign-in request.
10. Auth endpoint must reject invalid body with 400.
11. Failed smoke check exits non-zero for fast feedback.

System behavior and data:

- Health route is public and used for connectivity checks.
- Smoke test protects against accidentally public manager or exam routes.
- AWS setup docs explain SAM build/deploy and lab credentials.

Code trace:

- Script: `scripts/smoke.mjs`
- Backend: `backend/src/handlers/health.js`
- Infrastructure: `backend/template.yaml` (`HealthFunction`)
- Documentation: `README.md`, `docs/aws-lab-setup.md`, `docs/bootstrap-guide.md`, `docs/milestones.md`

Cross-Feature Data and Service Trace

- Cognito: F-01, F-02, F-03, F-04.
- DynamoDB Users: F-01, F-02, F-04, F-05, F-18.
- DynamoDB Personas: F-06, F-08, F-10.
- DynamoDB Scenarios: F-07, F-08, F-09, F-10, F-11, F-12, F-13.
- DynamoDB ExamSessions: F-13, F-14, F-15, F-16, F-17, F-18.
- SQS ExamIssueReleaseQueue: F-13, F-14.
- AWS Secrets Manager: F-10, F-16.
- OpenAI Responses API: F-10, F-16.
- React Router and route guards: F-02, F-04, F-05 through F-18.
- TanStack Query: F-05 through F-18 for loading, cache refresh, and optimistic local updates.

Main End-to-End Product Scenarios

Manager Builds Training Content

1. Manager signs in.
2. Manager creates or updates personas.
3. Manager creates a scenario draft with title, description, persona, and issue count.
4. Manager publishes the scenario.
5. Manager generates AI issues.
6. Manager reviews and edits issues.
7. Scenario becomes ready for reps when published and generated issue count matches issue target.

Rep Completes Exam and Receives Coaching

1. Rep signs in.
2. Rep selects a published scenario.
3. System creates a timed session and schedules issue arrivals.
4. Rep responds to visible customer issues.
5. Rep marks issues done.
6. Timer ends after 3 minutes.
7. System creates AI evaluation.
8. Rep reviews score, rubric, strengths, growth areas, practice ideas, and per-issue feedback.

Manager Reviews Outcomes

1. Manager opens dashboard.
2. System aggregates exam session and evaluation records.
3. Manager filters by scenario.
4. Manager reviews pass rate, average score, completion, reps needing evaluation, and coaching focus.
5. Manager exports CSV for offline reporting.

Notes for Final Submission

- Place this PDF inside folder 05 in the final ZIP/7Zip submission.
- Use the feature IDs F-01 through F-19 consistently in any code comments requested by the course instructions.
- This document describes implemented behavior from current source code and docs, not future backlog items.